CAMPUS EVENT MANAGEMENT SYSTEM

A PROJECT REPORT

SUBMITTED BY

ROLL NUMBER	REGISTRATION NUMBER	STUDENT CODE	STUDENT NAME
26102410051	26102000173	BWU/MCA/26/056	MIHIR ROY

in partial fulfillment for the award of the degree

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



ABSTRACT

The Campus Event Management System is a local web application developed to organize campus events and student participation through separate student and administrator access. Students can create accounts, sign in, browse and search events, register or cancel registrations, view their registration records, and submit event ratings with comments. Administrators can manage events, review student and registration records, moderate feedback, and view event participation and rating summaries.

The application is implemented primarily in Python, HTML and CSS. Python's built-in http.server module with ThreadingHTTPServer handles browser requests. The project is organized into request handling, model, service and analytics modules. CSV files store students, administrators, events, registrations and feedback. Pandas and NumPy are used for data analysis, while Matplotlib generates PNG charts. A small amount of inline JavaScript in the HTML templates switches the login and registration form labels between student and administrator roles.

The registration service checks student validity, event date, duplicate active registration and available capacity before saving a record. Registration is rejected when the event date is earlier than the current date or when active registrations reach the event limit; cancelled registrations are excluded from the capacity count. Feedback requires an active registration and is limited to one review per student per event. Administrators can edit or delete submitted reviews.

The source package contains Python modules, HTML templates, a CSS stylesheet, CSV data files, chart images and setup instructions. Static compilation of the 17 Python files completed successfully during documentation preparation. Source inspection also identified two implementation issues to be corrected: the daily feedback chart is generated but is not included in the HTTP chart allowlist, and the optional seed script calls the student-creation function without its required password argument when it needs to create demo students.

The project demonstrates modular Python programming, object-oriented design, input validation, exception handling, CSV file handling, password hashing, data processing and graphical presentation. Passwords for newly registered accounts are stored as salted PBKDF2-HMAC-SHA256 hashes; the built-in demonstration administrator is checked against fixed demo credentials defined in the source. The application is intended for local academic demonstration and can be extended with a database, email verification and secure deployment.

TABLE OF CONTENTS

Chapter	Title	Page No.
	ABSTRACT	2
	LIST OF TABLES	5
	LIST OF FIGURES	6
1.	**INTRODUCTION**	1
	1.1 Background	1
	1.2 Problem Statement	1
	1.3 Scope of the Project	1
	1.4 Main Modules	1
2.	**OBJECTIVE**	2
	2.1 Primary Objective	2
	2.2 Secondary Objectives	2
	2.3 Functional Objectives	2
	2.4 Technical Objectives	2
	2.5 User-centric Objectives	2
	2.6 Administrative Objectives	2
	2.7 Long-term Objectives	2
3.	**PLANNING**	3
	3.1 Need for Planning	3
	3.2 Project Development Methodology	3
	3.3 Work Breakdown Structure (WBS)	3
	3.4 Project Scheduling	4
	3.5 Module Identification	4
	3.6 Resource Planning	4
	3.7 Risk Analysis and Mitigation	4
4.	**REQUIREMENT ANALYSIS**	5
	4.1 User Requirements	5
	4.2 Functional Requirements	5
	4.3 Non-Functional Requirements	5
	4.4 System Requirements	6
	4.5 Data Requirements	6
	4.6 Existing System and Comparison	6
	4.7 Constraints	6
	4.8 Assumptions	6

Chapter	Title	Page No.
5.	**SYSTEM FLOW**	7
	5.1 Data Flow Diagram – Level 0	7
	5.2 Data Flow Diagram – Level 1	8
	5.3 Data Flow Diagram – Level 2	9
	5.4 Use Case Diagram	10
	5.5 Entity–Relationship Diagram	11
	5.6 Activity Diagram	12
	5.7 Flowcharts	13
6.	**PROPOSED DESIGN**	14
	6.1 Design Approach	14
	6.2 System Architecture Design	14
	6.3 User Interface (UI) Planning	14
	6.4 Navigation Design	15
	6.5 Data / Module Design	15
	6.6 Design Considerations	15
7.	**EXPERIMENTAL RESULT**	16
	7.1 Overview of Experimental Setup	16
	7.2 Authentication Results	16
	7.3 Event and Registration Results	16
	7.4 Feedback Management Results	16
	7.5 Analytics and Dashboard Results	16
	7.6 Graphical Analysis Results	17
	7.7 Performance Observations	18
	7.8 Testing Results and Code Review	18
8.	**FUTURE SCOPE**	19
9.	**CONCLUSION**	20
	APPENDICES	21
	Appendix A: User Interface Screens	21
	Appendix B: Sample Source Code	24
	Appendix C: CSV Data Structure	25
	Appendix D: Test Cases	26
	REFERENCES	27

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Main modules of the Campus Event Management System	1
Table 3.1	Project development methodology	3
Table 3.2	Project schedule	4
Table 3.3	Resource Planning	4
Table 3.4	Risk Analysis & Mitigation	4
Table 4.1	Functional requirements	5
Table 4.2	Non-functional requirements	5
Table 4.3	System requirements	6
Table 4.4	Data requirements	6
Table 4.5	Existing System and Comparison	6
Table 6.1	Navigation Design	15
Table 6.2	Data / Module Design	15
Table 7.1	Testing Results and Code Review	18
Table C.1	CSV data structure	25
Table D.1	Project test cases	26

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 5.1	Data Flow Diagram – Level 0	7
Figure 5.2	Data Flow Diagram – Level 1	8
Figure 5.3	Data Flow Diagram – Level 2: Registration Validation	9
Figure 5.4	Use Case Diagram	10
Figure 5.5	Entity–Relationship Diagram	11
Figure 5.6	Activity Diagram – Registration	12
Figure 5.7	Feedback Submission Flowchart	13
Figure 6.1	System Architecture	14
Figure 7.1	Event-wise Participation Chart	17
Figure 7.2	Average Rating by Event Chart	17
Figure 7.3	Rating Distribution Chart	17
Figure 7.4	Daily Feedback Statistics Chart	18

CHAPTER 1

INTRODUCTION

College campuses organize workshops, seminars, cultural programmes, sports meets and other activities throughout the academic year. Coordinating these events requires accurate event details, participant records, registration limits and feedback from students. Manual notices and spreadsheets can make it difficult to keep information updated and to prevent duplicate or excessive registrations.

The Campus Event Management System provides a centralized local web application for students and administrators. Students can create accounts, browse events, register, cancel registrations, view their records and submit feedback. Administrators can maintain event details, review student and registration records, moderate feedback and inspect analytical summaries.

1.1 Background

The project was developed as a syllabus-oriented Python application. It uses Python's built-in HTTP server instead of a web framework. HTML templates provide the page structure, CSS defines the visual layout, and CSV files store application records. Pandas, NumPy and Matplotlib are used for analysis and charts.

1.2 Problem Statement

The project addresses the need for a simple campus platform that organizes event information and student participation. It must validate account details, prevent duplicate registrations, stop registrations for past events or full events, preserve student and event records, and provide a way to collect and review feedback.

1.3 Scope of the Project

- Student and administrator account creation and password-based login.
- Event creation, editing, deletion, browsing, searching and date filtering.
- Student registration and cancellation with date, capacity and duplicate checks.
- Student registration history and administrator registration monitoring.
- Feedback ratings from 1 to 5 with written comments, limited to eligible registered students.
- Administrator search, edit and deletion of student reviews.
- Dashboard statistics, event participation, event ratings and daily feedback chart generation.
- CSV-based persistence and PNG chart generation using Python libraries.

1.4 Main Modules

Module	Description
Authentication	Student/admin sign-up, password hashing, login, session creation and role checks.
Event Management	Create, read, update, delete, search and filter event records.
Registration	Register or cancel; validate event date, participant capacity and duplicate active registrations.
Feedback	Submit one rating and comment per registered student/event; validate and store review data.
Administrator Management	Review student records, registrations and feedback; edit or delete feedback.
Analytics	Calculate dashboard statistics, highest/lowest rated events and chart data.
File Management	Initialize, read and write CSV files using the csv module.

CHAPTER 2

OBJECTIVE

2.1 Primary Objective

To develop a simple and reliable campus event management application that enables students to discover and register for campus events while allowing administrators to maintain event records and evaluate student feedback.

2.2 Secondary Objectives

- Reduce manual effort in event and participant record management.
- Provide separate student and administrator access.
- Prevent registration after an event date has passed or when the event is full.
- Maintain consistent registration and feedback records in CSV files.
- Present event ratings and daily feedback information through charts.

2.3 Functional Objectives

- Allow students and administrators to create accounts and sign in using role-specific information and passwords.
- Allow students to browse, search and filter events and view their registrations.
- Allow administrators to create, update and delete events and search student and registration records.
- Allow students with an active registration to submit one review per event.
- Allow administrators to search, edit and delete submitted feedback.

2.4 Technical Objectives

- Use Python and the built-in ThreadingHTTPServer for request handling and business logic.
- Use HTML templates and CSS for the browser interface, with small inline JavaScript for role switching.
- Use CSV files for local data persistence.
- Use Pandas and NumPy for data analysis and Matplotlib for chart generation.
- Use Python classes, modular functions, exception handling, input validation and PBKDF2 password hashing.

2.5 User-centric Objectives

Provide readable forms, clear navigation, helpful validation messages and visible event dates and participant limits. Students should be able to understand whether an event is available and whether they have already registered.

2.6 Administrative Objectives

Provide administrators with tools to manage event records, inspect student and registration information, moderate submitted reviews and identify events with higher or lower average ratings.

2.7 Long-term Objectives

The project can later be extended with a relational database, institutional email verification, password recovery, event notifications, attendance tracking, downloadable reports and secure deployment on a university server.

CHAPTER 3

PLANNING

3.1 Need for Planning

Planning helps identify the required user roles, define the event and feedback workflows, select data fields, divide the code into modules and schedule development and testing. It also helps identify risks such as invalid credentials, duplicate registration, full events and inconsistent CSV data.

3.2 Project Development Methodology

The project follows an incremental development approach. Requirements and interface needs are identified first. Data models and CSV files are then prepared, followed by authentication, event and registration services, feedback management, analytics, integration and testing.

Phase	Activity	Output
1. Requirement study	Identify student/admin tasks and business rules.	Requirement list
2. Design	Plan modules, navigation, data fields and diagrams.	System design
3. Implementation	Develop Python services, templates and CSS.	Working modules
4. Integration	Connect request handler, services, CSV storage and charts.	Integrated application
5. Testing	Review authentication, event, registration and feedback rules.	Test results
6. Documentation	Prepare diagrams, screenshots, source notes and report.	Project report

3.3 Work Breakdown Structure (WBS)

1. Project initiation and requirement collection.
2. Design of student and administrator workflows.
3. CSV file and model design.
4. Authentication and password management.
5. Event management and registration services.
6. Feedback moderation and analytics.
7. Chart generation, integration and testing.
8. Documentation and final review.

3.4 Project Scheduling

Activity	Week 1	Week 2	Week 3	Week 4
Requirement and design	✓			
Authentication and CSV files	✓	✓		
Event and registration modules		✓	✓	
Feedback and analytics			✓	
Integration and testing			✓	✓
Documentation				✓

3.5 Module Identification

The source code is divided into request handling, configuration, models, services, analytics, HTML templates, static CSS and data files. The separation allows the application to keep page rendering, validation, file operations and statistical calculations in separate components.

3.6 Resource Planning

Resource	Purpose
Python 3.10+	Application logic and built-in HTTP server.
Visual Studio Code	Code editing and debugging.
HTML	Page structure and form elements.
CSS	Layout, colours, responsive styling and controls.
Inline JavaScript	Switch login and sign-up labels for student/admin roles.
CSV files	Local storage for students, admins, events, registrations and feedback.
Pandas / NumPy	Data processing and numerical calculations.
Matplotlib	Generation of event and feedback charts.
Web browser	Run and test the user interface.

3.7 Risk Analysis and Mitigation

Risk	Mitigation
Invalid account details	Validate names, IDs, Gmail addresses and passwords.
Duplicate registration	Check for an existing active registration before saving.
Event capacity exceeded	Count active registrations before accepting a new record.
Past event registration	Compare the event date with the current date in Python.
Password exposure	Store salted PBKDF2-HMAC-SHA256 hashes instead of plain text.
CSV inconsistency	Initialize required headers and handle file read/write exceptions.

CHAPTER 4

REQUIREMENT ANALYSIS

4.1 User Requirements

- Students need to create an account using their name, student code, Gmail address and password.
- Students need to browse events, register for available events and cancel their own registrations.
- Students need to view registration history and submit feedback for an event for which they have an active registration.
- Administrators need to create, update and delete events and review student and registration records.
- Administrators need to search, edit and delete feedback and view event and feedback analytics.

4.2 Functional Requirements

ID	Functional requirement
FR-01	The system shall support student account creation and password-based login.
FR-02	The system shall support administrator login and role-protected administrator pages.
FR-03	The administrator shall add, edit and delete event records.
FR-04	Users shall browse, search and filter events.
FR-05	The system shall reject registration when the event date is earlier than the current date.
FR-06	The system shall reject registration when active registrations reach the event capacity.
FR-07	The system shall prevent duplicate active registration by the same student for the same event.
FR-08	Students shall view and cancel their own active registrations.
FR-09	Eligible students shall submit one rating and comment per event.
FR-10	Administrators shall search, edit and delete feedback.
FR-11	The system shall calculate dashboard statistics and generate charts.

4.3 Non-Functional Requirements

Attribute	Description
Usability	Clear navigation, readable forms and understandable validation messages.
Reliability	Business rules are checked in Python before data is saved.
Security	Password hashing, session cookies and role checks protect application pages.
Maintainability	Models, services, analytics, templates and CSS are separated.
Performance	CSV storage and chart refresh are suitable for a small local academic demonstration.
Portability	Runs on a computer with Python and the listed dependencies installed.

4.4 System Requirements

Category	Requirement
Hardware	Desktop/laptop computer, keyboard, mouse and sufficient memory for Python and browser.
Software	Windows/Linux/macOS with Python 3.10 or later.
Editor	Visual Studio Code recommended.
Browser	Modern web browser.
Libraries	Pandas, NumPy and Matplotlib as listed in requirements.txt.
Server	Python built-in http.server with ThreadingHTTPServer, local host 127.0.0.1 and port 8000.

4.5 Data Requirements

The project uses five CSV files. The file service creates the required files and headers if they do not exist. The application does not use MySQL, SQLite or another database in this version.

CSV file	Purpose / principal fields
students.csv	student_id, name, email, student_code, password_hash
admins.csv	admin_id, name, email, admin_code, password_hash
events.csv	event_id, event_name, date, venue, max_participants, description
registrations.csv	registration_id, student_id, event_id, registration_date, status
feedback.csv	feedback_id, student_id, event_id, rating, comment, feedback_date

4.6 Existing System and Comparison

Manual approach	Proposed application
Notices and separate spreadsheets	Central event listing and search.
Participant count updated manually	Active registration count and capacity validation.
Duplicate rows may occur	Duplicate active registration is rejected.
Feedback collected separately	Rating and comment records are stored with event/student IDs.
Manual review summary	Dashboard statistics and Matplotlib charts.

4.7 Constraints

The application is designed for local academic use and uses CSV files rather than a database. It runs on the local host by default. The session store is held in memory and is cleared when the server process stops. Email validation is restricted to Gmail addresses in the authentication service.

4.8 Assumptions

Users have access to a browser and the application is started through app.py. Administrators maintain accurate event dates and participant limits. CSV files remain available and writable. The included demo credentials are for classroom demonstration and should be changed before any wider use.

CHAPTER 5

SYSTEM FLOW

5.1 Data Flow Diagram – Level 0

The Level 0 DFD divides the system into authentication, event and registration processing, and feedback and analytics. Each process communicates with the appropriate CSV data files.

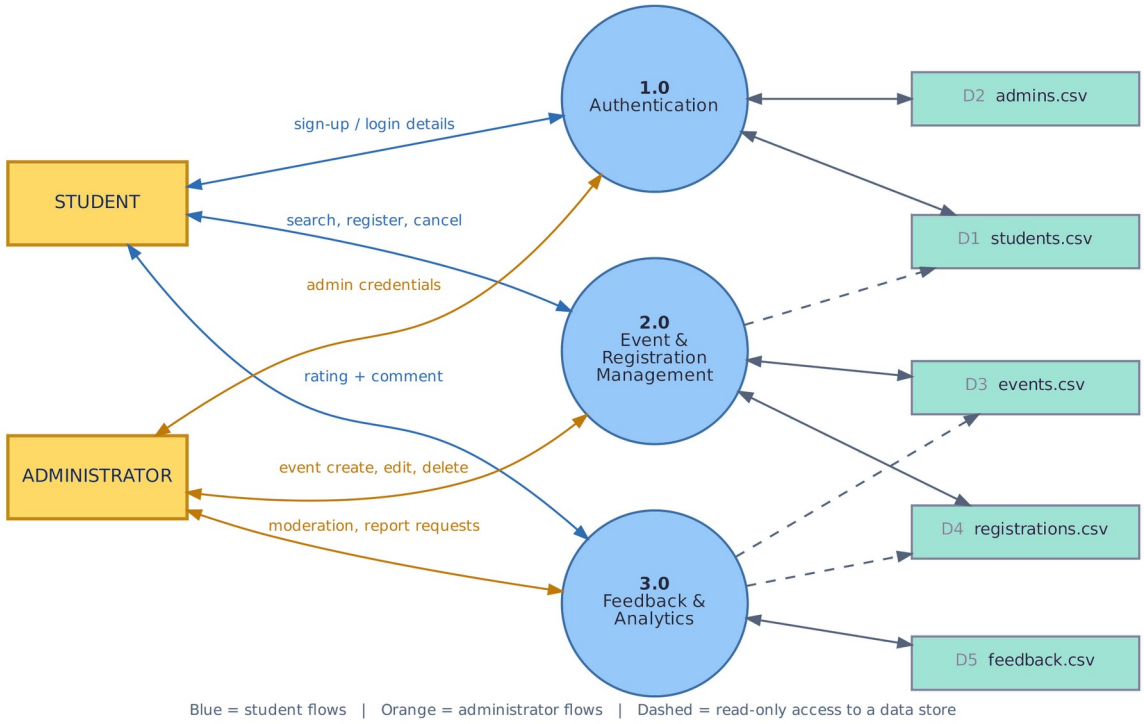


Figure 5.1 – Data Flow Diagram Level 0

5.2 Data Flow Diagram – Level 1

The Level 1 diagram expands the event registration workflow. The student selects an event, the system reads event information, checks date and capacity, and saves the registration only when all validations pass.

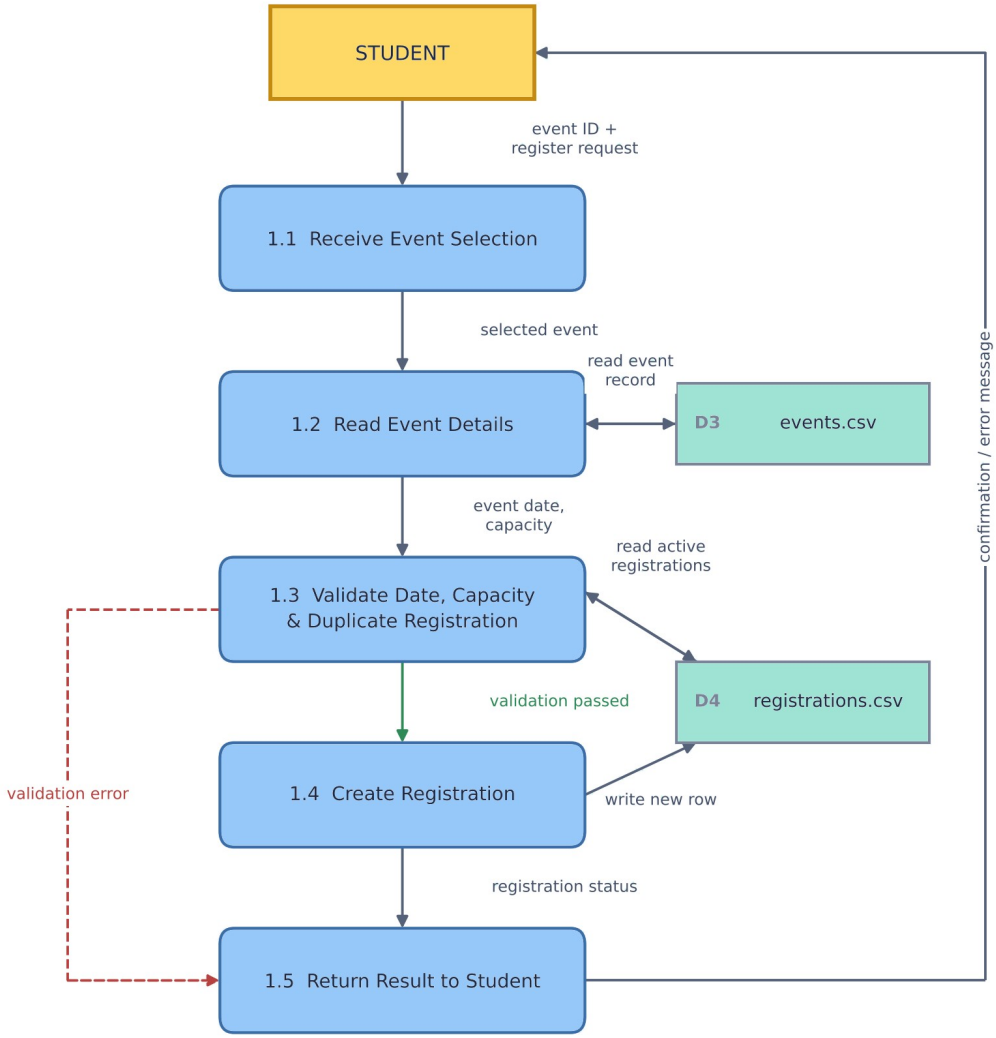


Figure 5.2 – Data Flow Diagram Level 1

5.3 Data Flow Diagram – Level 2

The Level 2 diagram decomposes registration validation into student verification, event lookup, date and duplicate checks, capacity validation and registration storage. The sequence follows the registration service implementation.

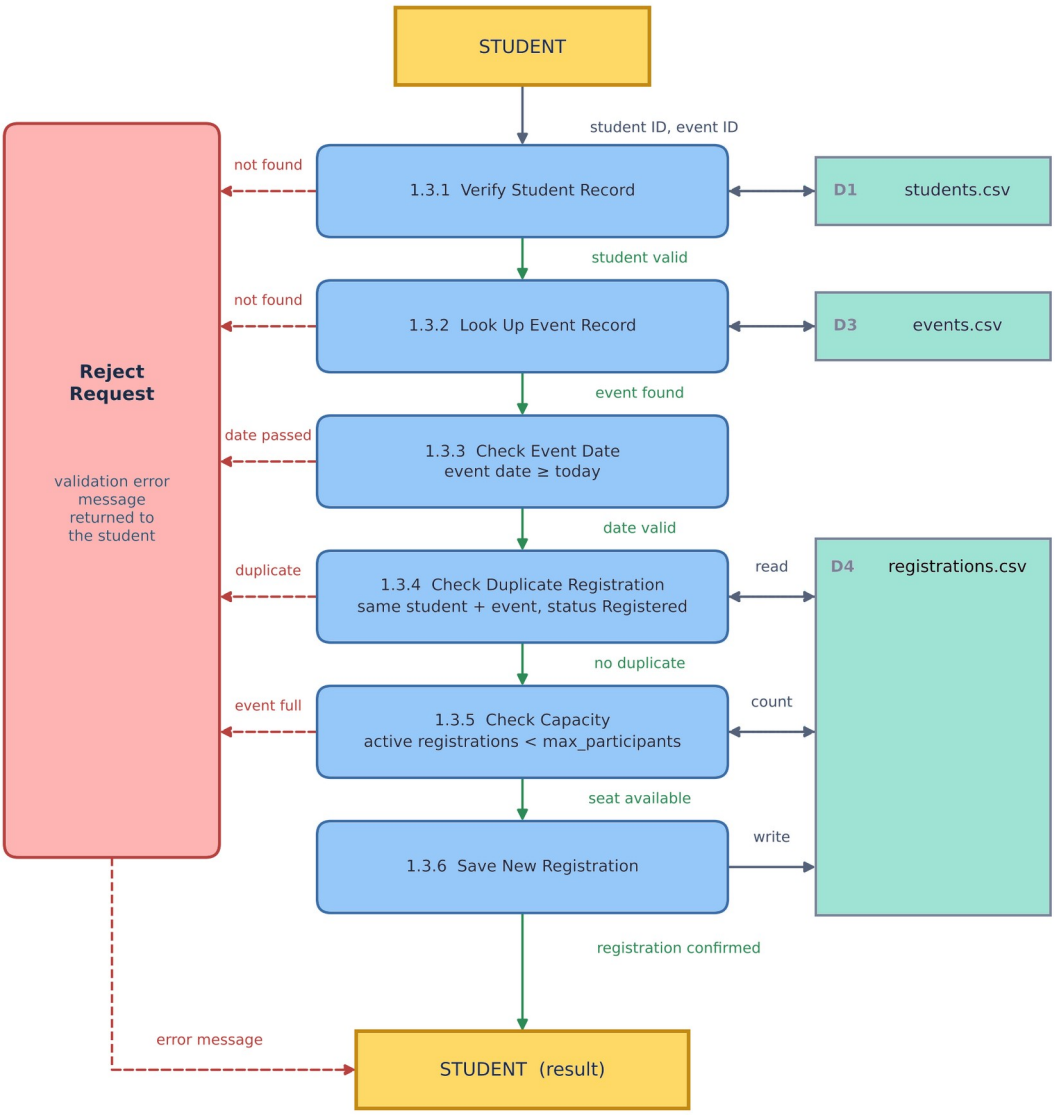


Figure 5.3 – Data Flow Diagram Level 2: Registration Validation

5.4 Use Case Diagram

The student actor can create an account, sign in, browse events, register, cancel a registration, view registration records and submit feedback. The administrator actor can sign in, manage events, inspect students and registrations, moderate feedback and view analytics.

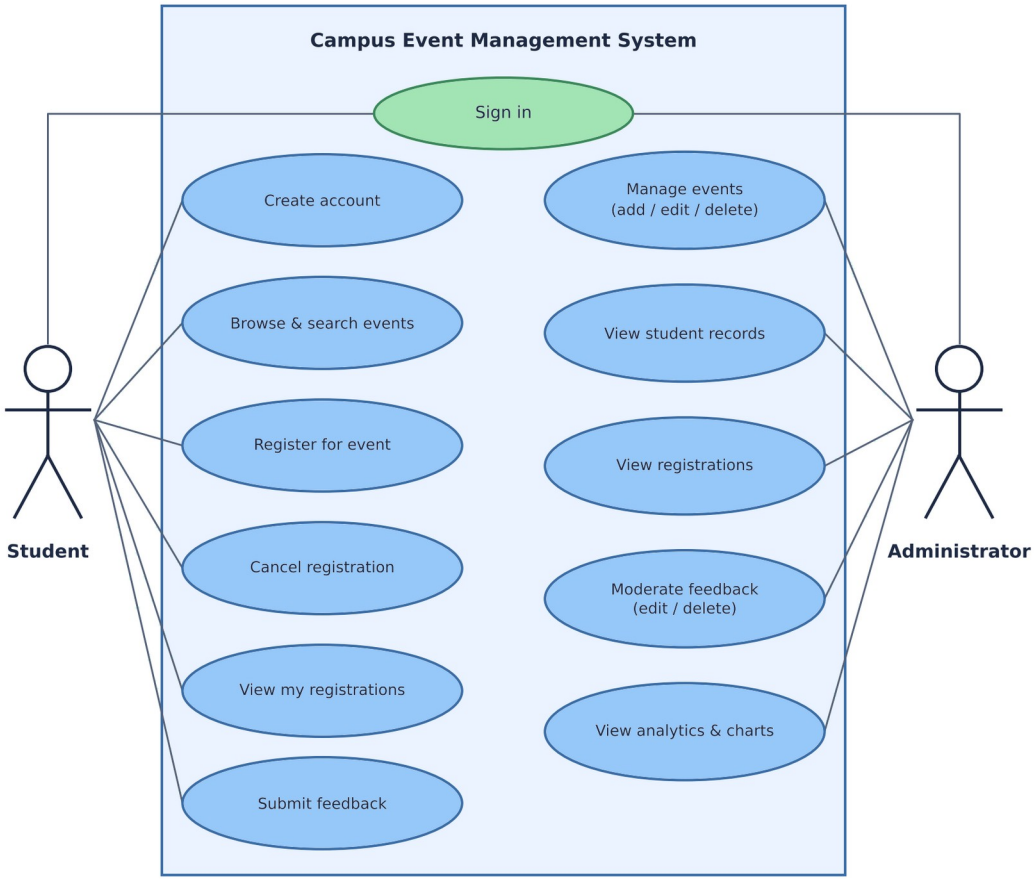


Figure 5.4 – Use Case Diagram

5.5 Entity–Relationship Diagram

The logical data model contains Student, Admin, Event, Registration and Feedback entities. A student can have multiple registration records and feedback records. Each registration and feedback record references one event and one student.

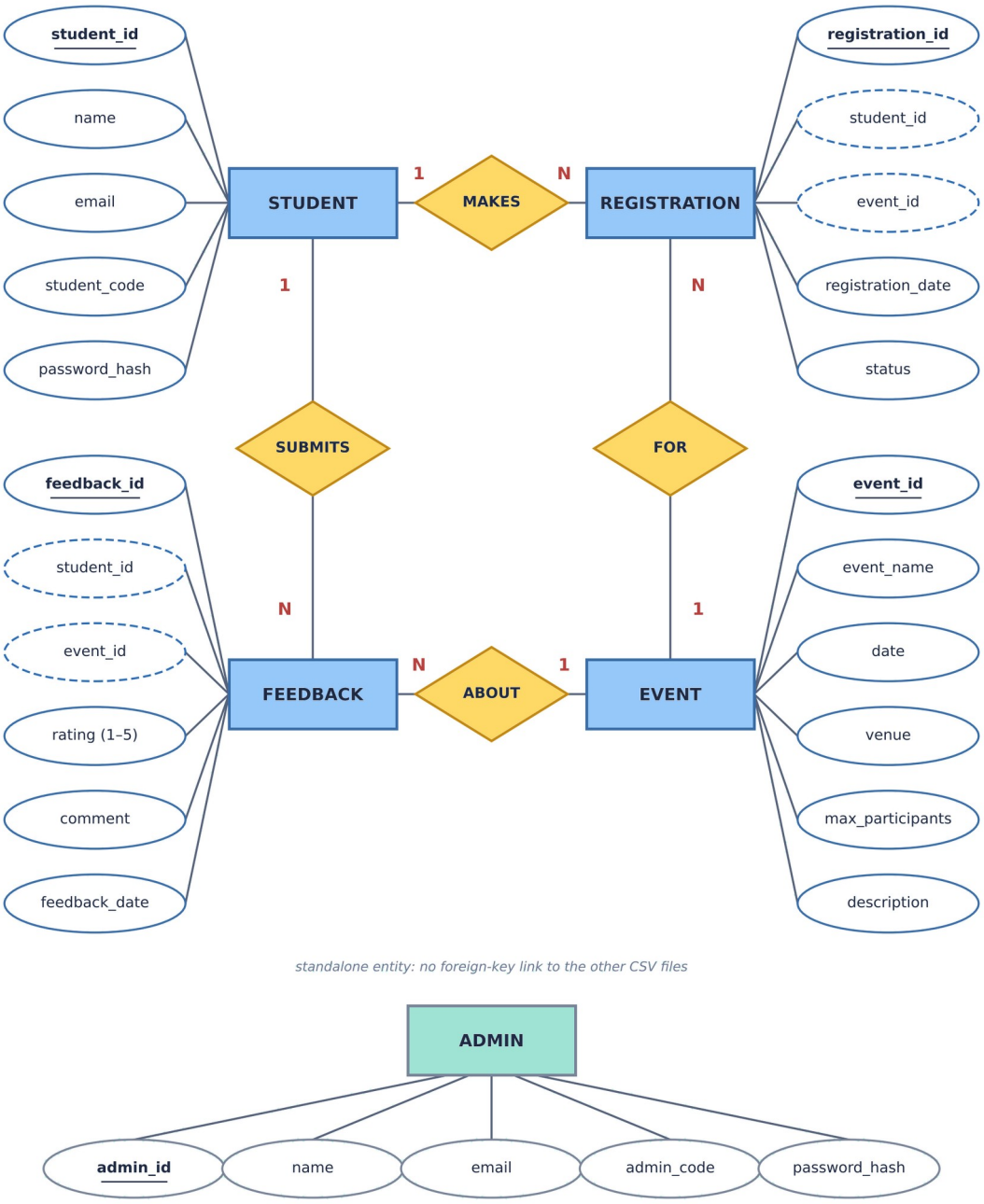


Figure 5.5 – Entity–Relationship Diagram

5.6 Activity Diagram

The registration activity begins when a student selects an event. The application verifies the student, reads the event date, checks for an existing active registration, counts active registrations and compares the count with the event capacity. If validation succeeds, a new record is saved.

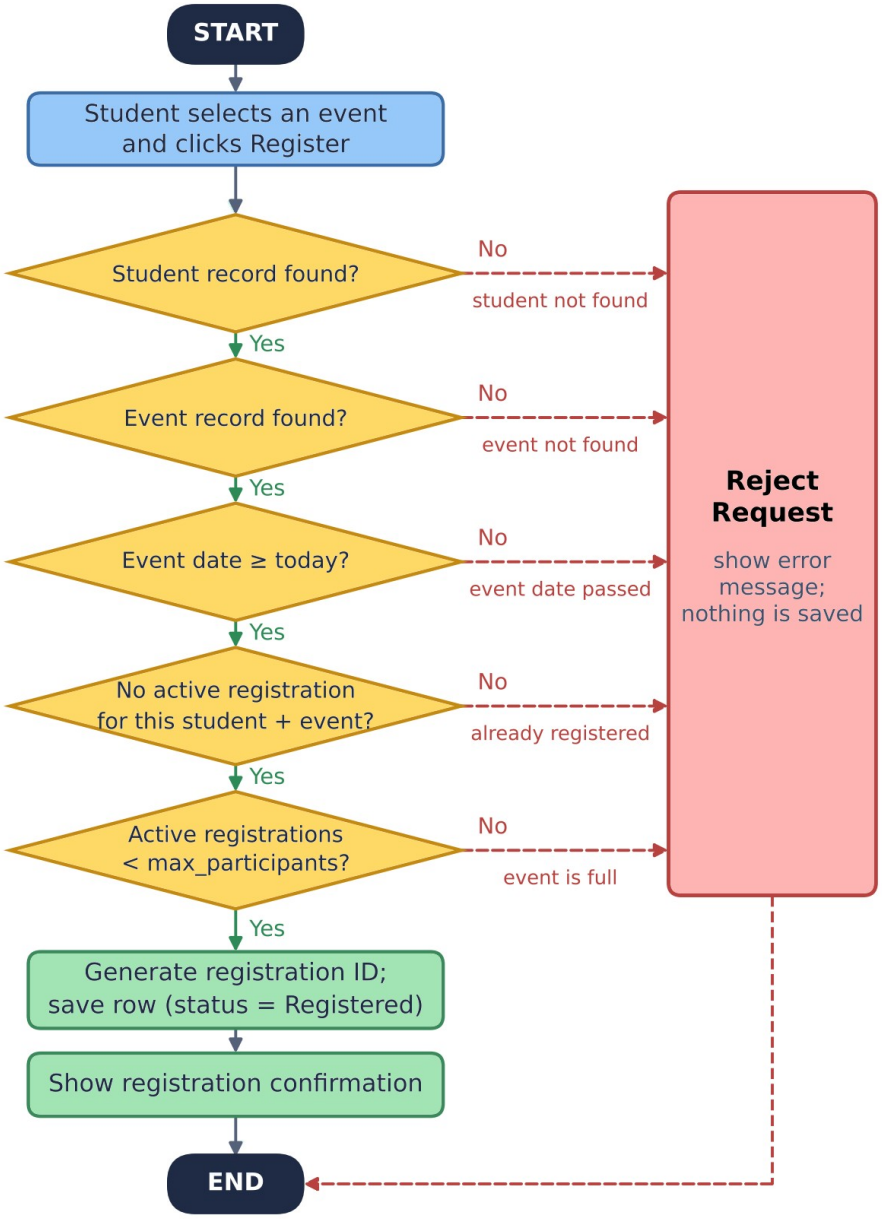


Figure 5.6 – Activity Diagram: Registration

5.7 Flowcharts

The feedback flowchart shows that feedback is accepted only if the selected event exists, the student has an active registration and no feedback record already exists for that student/event pair. The rating is then validated and the feedback row is stored.

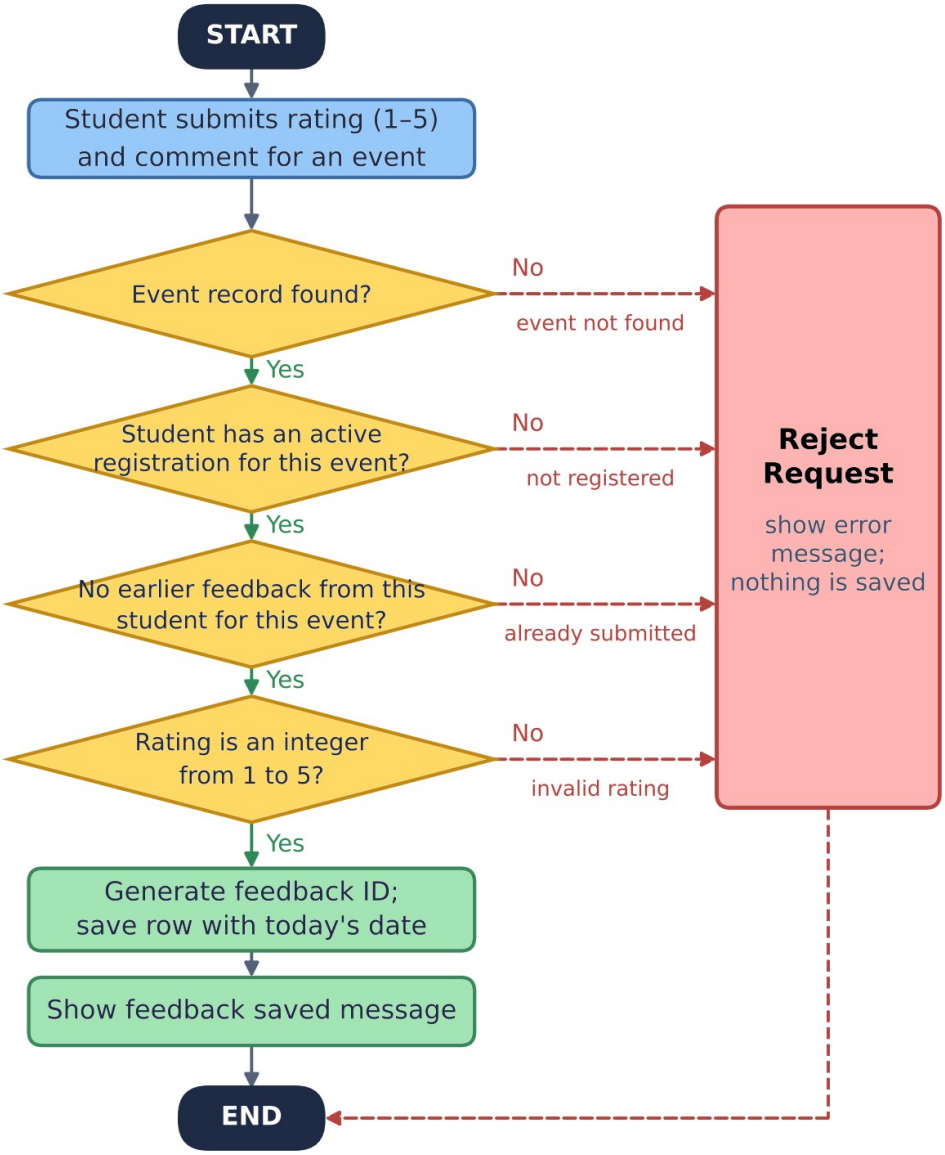


Figure 5.7 – Feedback Submission Flowchart

CHAPTER 6

PROPOSED DESIGN

6.1 Design Approach

The application uses a modular design. The request handler coordinates page requests and form submissions. Model classes represent student, event, registration and feedback records. Service modules implement authentication, file operations, event management, registration and feedback rules. Analytics modules calculate statistics and generate charts.

6.2 System Architecture Design

A browser sends HTTP requests to the Python ThreadingHTTPServer. The Handler class routes requests, checks the session and user role, calls the relevant service, and renders HTML. Services read or write CSV files through file_service.py. Analytics reads the records through Pandas and NumPy, while Matplotlib saves chart images in the charts folder.

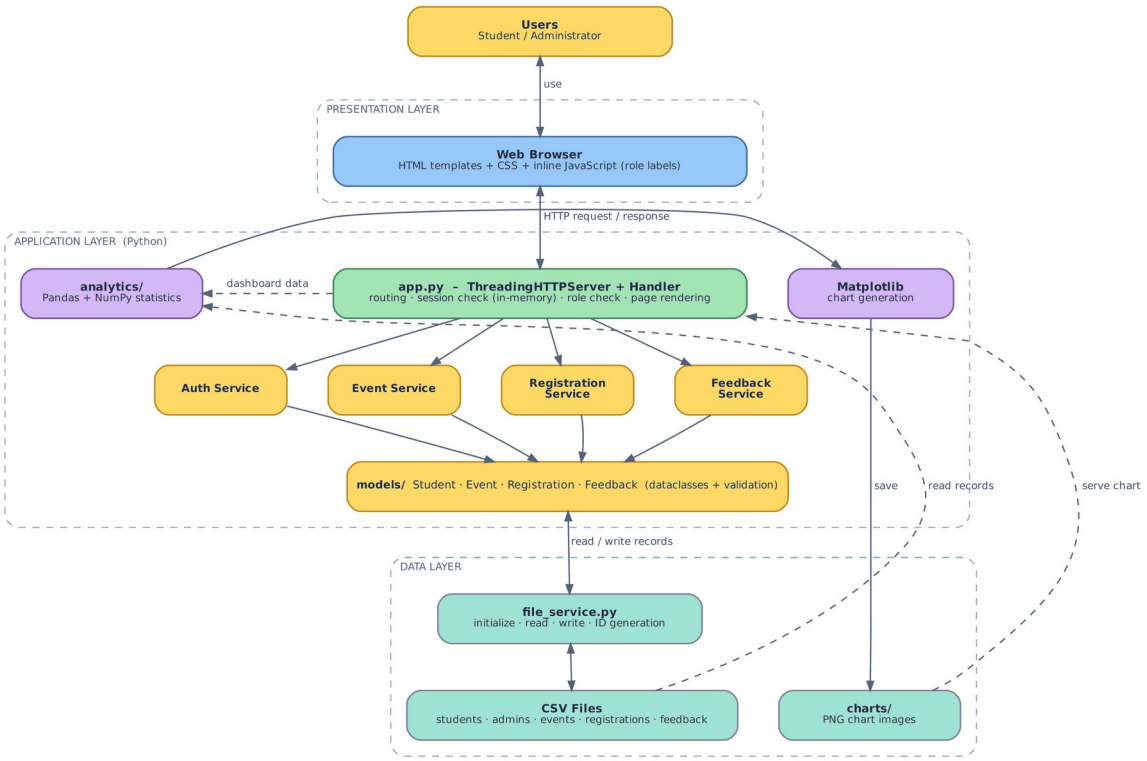


Figure 6.1 – System Architecture

6.3 User Interface (UI) Planning

- Login page: separate Student Login and Admin Login choices with name, ID, Gmail address and password fields.
- Sign-up page: role-specific student or administrator labels and input placeholders.
- Student dashboard: registration count, completed events, feedback count, available events and featured events.
- Browse Events: search by event name, ID or venue and filter by date.
- My Registrations: show registration ID, event, date, status and cancellation action.
- Administrator dashboard: event and registration summaries, highest/lowest rating and charts.
- Feedback workspace: search and rating filters, review cards, edit and delete controls, and daily feedback statistics.

6.4 Navigation Design

Student navigation	Administrator navigation
Dashboard	Dashboard
Browse Events	Manage Events
My Registrations	Students
Feedback	Registrations
Profile	Feedback & Analytics

6.5 Data / Module Design

Source file / folder	Responsibility
app.py	HTTP handler, routes, sessions and page rendering.
config.py	Project paths, local host, port and demo administrator settings.
models/	Dataclasses and validation for student, event, registration and feedback.
services/auth_service.py	Gmail/name/password validation, password hashing and account creation/login.
services/file_service.py	CSV headers, initialization, reading, writing and ID generation.
services/event_service.py	Event create/update/delete and event lookup.
services/registration_service.py	Registration, cancellation, date, capacity and duplicate checks.
services/feedback_service.py	Feedback submission, validation, update and deletion.
analytics/	Dashboard statistics and Matplotlib chart generation.
templates/ and static/	HTML pages and CSS stylesheet.

6.6 Design Considerations

Passwords are stored as salted PBKDF2-HMAC-SHA256 hashes. Session tokens are generated with Python's secrets module and stored in a process-local session dictionary. The application checks roles before administrator routes. Event deletion and student deletion are blocked when historical registrations or feedback exist. The current implementation is intended for a local demonstration, not production deployment.

CHAPTER 7

EXPERIMENTAL RESULT

7.1 Overview of Experimental Setup

The project source package was reviewed from the supplied ZIP file. It contains 17 Python files, three HTML templates, one CSS stylesheet, five CSV data files, generated chart images, requirements.txt and README.md. Python syntax compilation completed successfully for all 17 Python files during preparation of this report.

The README specifies Python 3.10 or later. The application is started from the project folder with python app.py and is accessed at http://127.0.0.1:8000. The requirements file lists Pandas, NumPy and Matplotlib.

7.2 Authentication Results

Student and administrator login use separate role choices. Both roles provide a name, ID, Gmail address and password. The authentication service checks the role and credentials. Passwords are hashed with a random 16-byte salt and PBKDF2-HMAC-SHA256 using 240,000 iterations. New passwords must contain at least eight characters, an uppercase letter, a lowercase letter and a number.

The built-in demonstration administrator is configured as Administrator / ADMIN001 / admin.demo@gmail.com with the demo password stated in the README. The README also identifies a demo student account. Demo credentials must be changed before any wider use.

7.3 Event and Registration Results

Administrators can create, edit and delete event records. The event service prevents a capacity value from being reduced below the number of active registrations. An event with registration or feedback history cannot be deleted. Students can search and filter events, register and cancel their own registrations.

The registration service rejects a registration when the event date is earlier than today's date, when an active registration already exists for the same student and event, or when the event has reached its participant limit. Registration remains possible on the event date itself. Cancelled registrations do not count toward capacity.

7.4 Feedback Management Results

A student can submit a rating from 1 to 5 and a comment for an event only when the student has an active registration. The service prevents a second feedback record for the same student and event. The administrator feedback page supports search by student, event or comment, rating filtering, review editing and review deletion.

The current service checks for an active registration but does not check whether the event date has passed. Therefore, the source code permits feedback submission before the scheduled event date if the student is already registered. This behaviour is documented as implemented and can be changed if post-event-only feedback is required.

7.5 Analytics and Dashboard Results

The analytics module calculates total events, active registrations, unique participants, total feedback, overall average rating, highest-rated event, lowest-rated event, event-wise registration counts and average ratings. Highest and lowest ratings are calculated from events that have feedback records.

The chart module generates event-wise participation, average rating by event, rating distribution and daily feedback counts for the latest 14 calendar days. The daily series includes dates with zero feedback.

7.6 Graphical Analysis Results

The following figures are chart files included in the supplied project ZIP. They represent the chart outputs generated by the analytics module from the current CSV data.

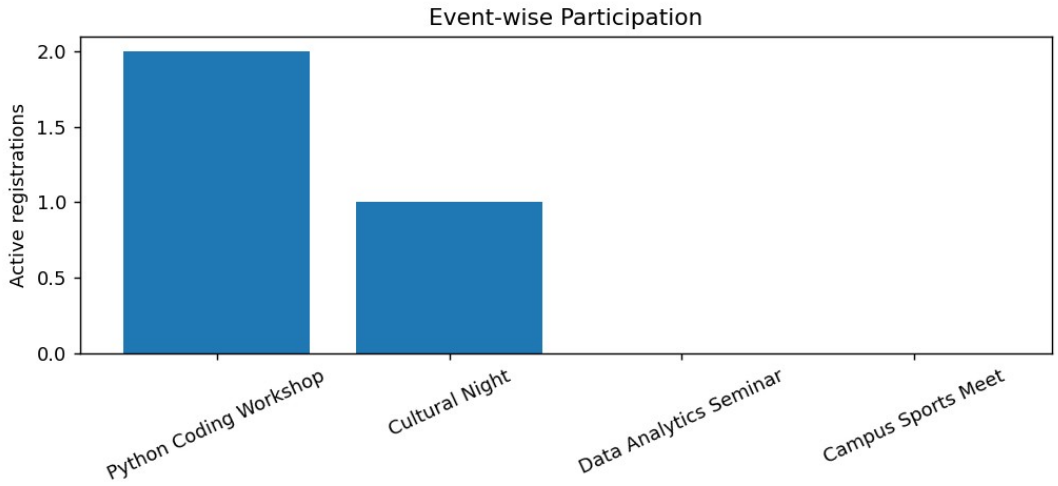


Figure 7.1 – Event-wise Participation Chart

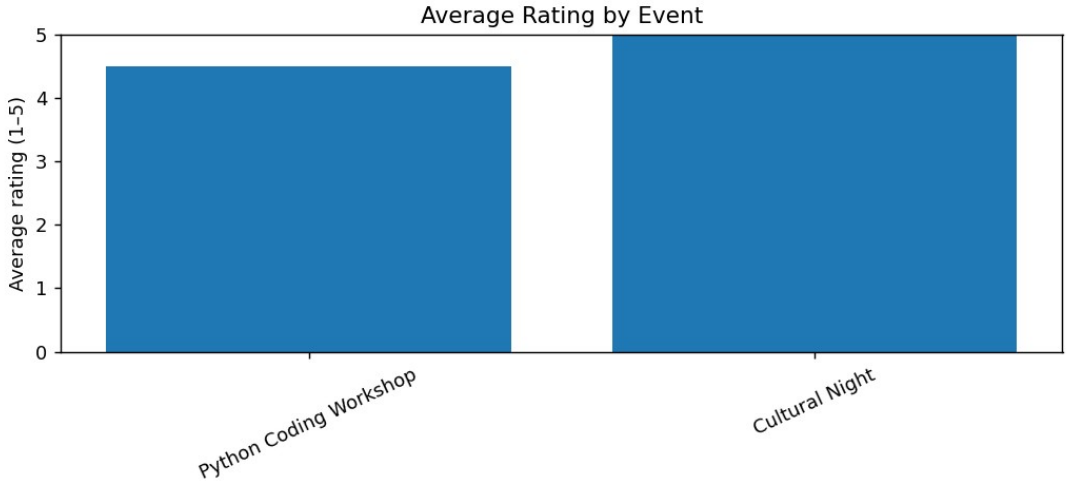


Figure 7.2 – Average Rating by Event Chart

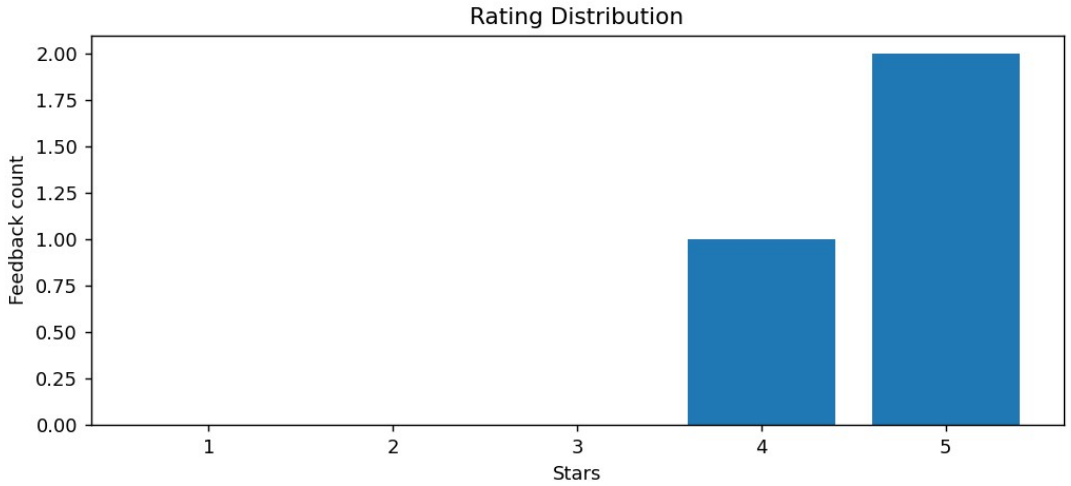


Figure 7.3 – Rating Distribution Chart

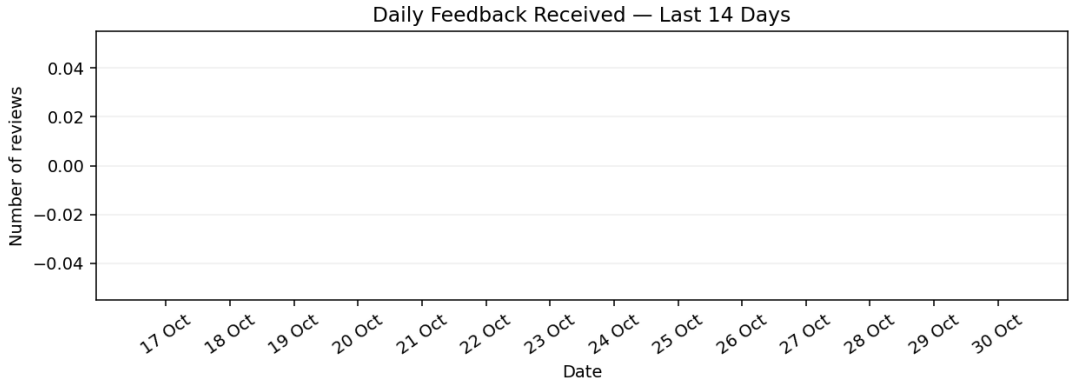


Figure 7.4 – Daily Feedback Statistics Chart (latest 14 days)

7.7 Performance Observations

The project uses local CSV files and generates charts as PNG images. This arrangement is straightforward for a small academic demonstration but is not designed for heavy concurrent traffic or large datasets. The session dictionary is stored in memory and is reset when the server restarts.

7.8 Testing Results and Code Review

Review item	Observed result
Python syntax	All 17 Python source files compiled successfully.
Registration date and capacity rules	Implemented in services/registration_service.py.
Duplicate registration check	Implemented for active registrations.
Feedback eligibility and duplicate check	Implemented in services/feedback_service.py.
Admin feedback edit/delete	Implemented through administrator POST routes.
Daily feedback chart access	Chart is generated and requested by the feedback page, but feedback_daily.png is missing from the allowed chart-name list in app.py; the browser request can return 404.
Optional seed script	seed_demo.py calls create_student without the required password argument if the student CSV is empty; the script needs correction for that case.

CHAPTER 8

FUTURE SCOPE

Database integration: Replace CSV files with SQLite or MySQL for better concurrent access and larger records.

Email verification: Verify student and administrator email addresses and provide account recovery.

Event notifications: Send reminders for upcoming events and registration closing dates.

Attendance tracking: Add QR-code check-in to record actual event participation.

Feedback policy: Allow feedback only after the scheduled event date, if required by the institution.

Analytics improvement: Add date-range filters, department-wise reports and downloadable PDF/Excel summaries.

Security enhancement: Add CSRF protection, secure cookie settings, stronger administrator provisioning and HTTPS deployment.

Deployment: Host the application on a secure university server with backups and production monitoring.

CHAPTER 9

CONCLUSION

The Campus Event Management System provides a structured way to publish campus events and manage student participation. It combines student and administrator access, event management, registration processing, feedback collection and analytical summaries in one local web application.

The supplied source code implements important validation rules, including password hashing, role checks, duplicate registration prevention, event-date validation and participant-capacity checks. It also provides administrator controls for event records, student records, registrations and feedback. CSV files provide persistent storage, while Pandas, NumPy and Matplotlib support data analysis and chart generation.

The source review confirms that the core Python files compile. It also identifies a chart-serving allowlist issue affecting the daily feedback image and a missing password argument in the optional demo seed script when the student file is empty. Correcting these issues will make the documented features more reliable in a fresh setup.

The project demonstrates practical use of Python, HTML, CSS, inline JavaScript, object-oriented programming, file handling, exception handling, data validation, password hashing, data analysis and visualization. It is suitable as an academic project and can be extended with a database, email verification, attendance tracking and secure deployment.

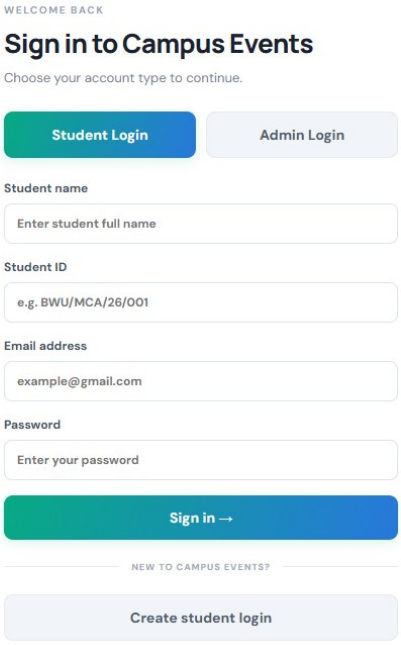
APPENDICES

APPENDIX A: USER INTERFACE AND CHART SCREENS

The supplied source ZIP contains generated chart images but does not contain captured browser screenshots of every page. The chart outputs below are inserted from the supplied project files. Add final screenshots from the running application in the indicated spaces before submission.

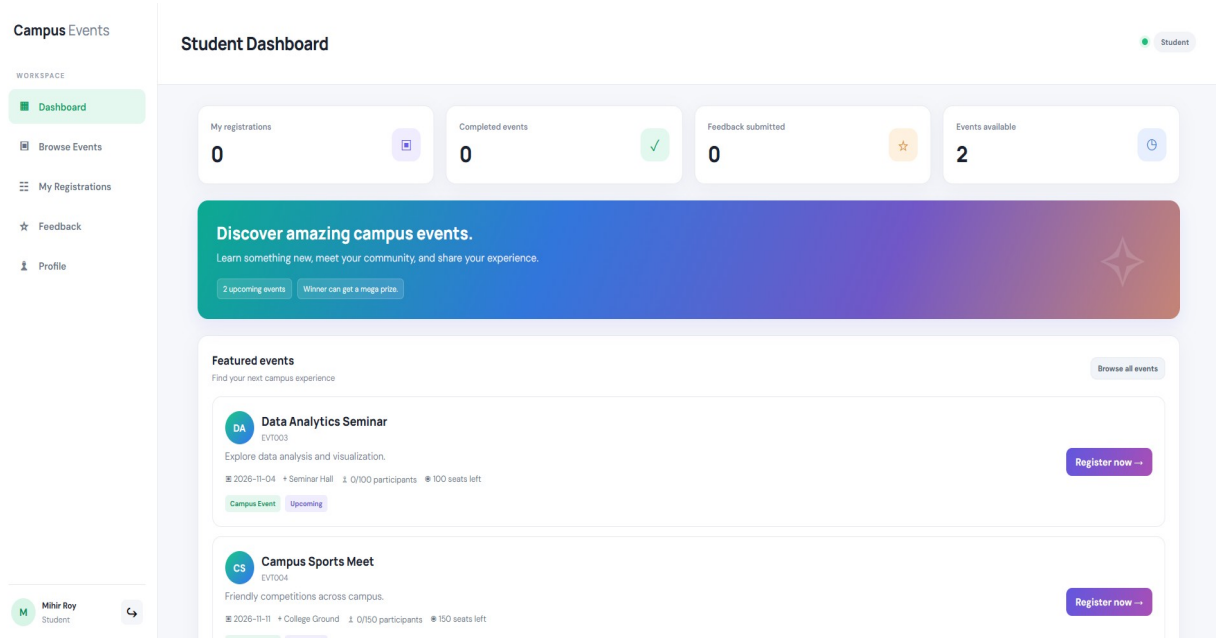
A.1 Login Page

Student and administrator login options with name, ID, Gmail address and password.



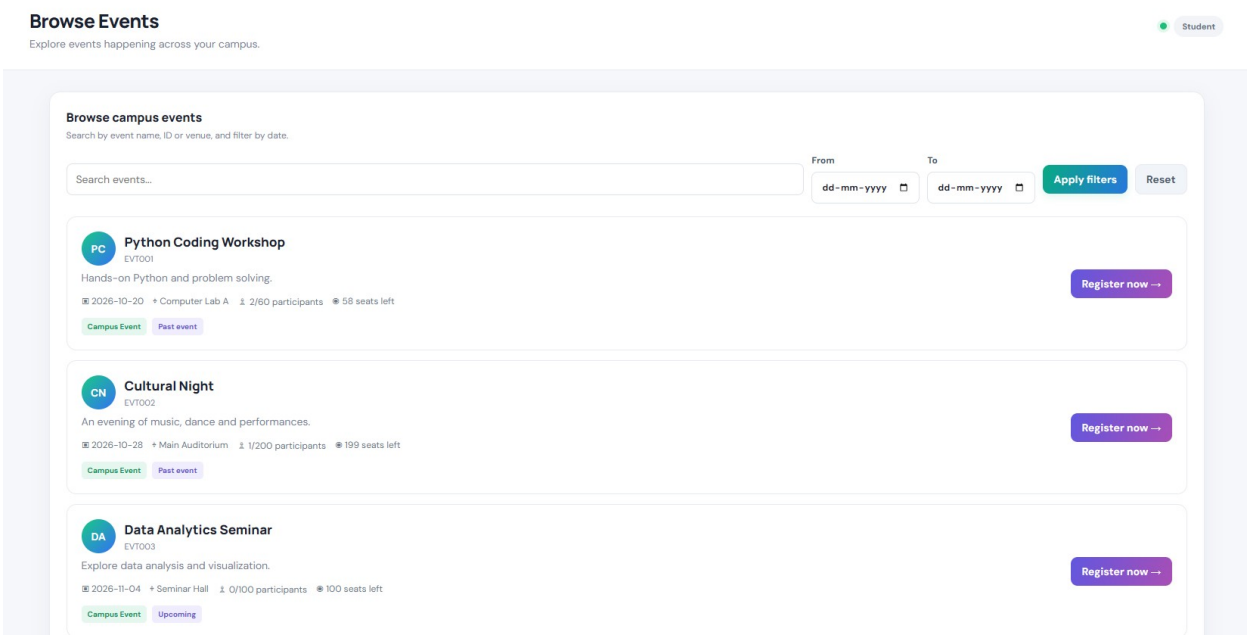
A.2 Student Dashboard

Student summary cards and featured campus events.



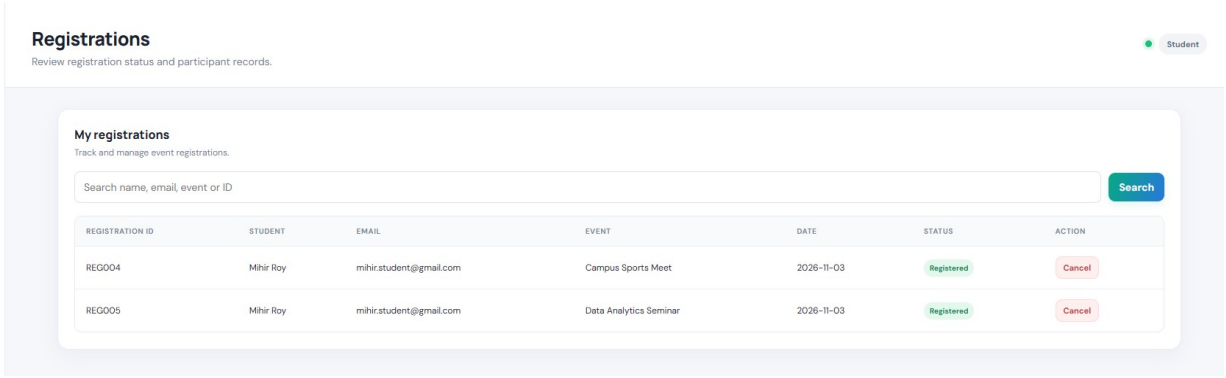
A.3 Browse Events

Event list, date filters, capacity and registration actions.



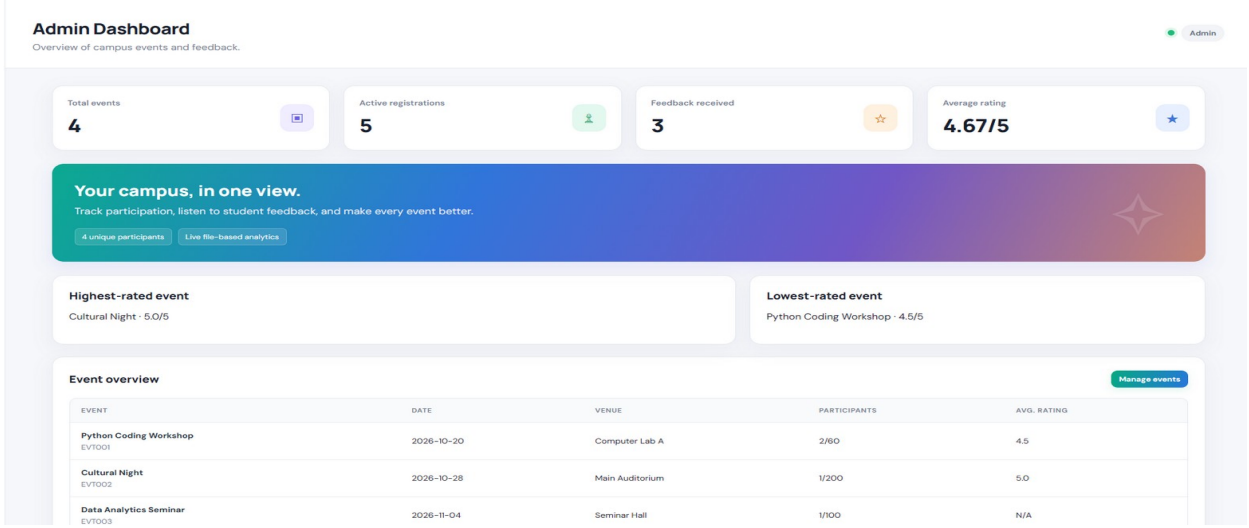
A.4 My Registrations

Student registration records and current status.



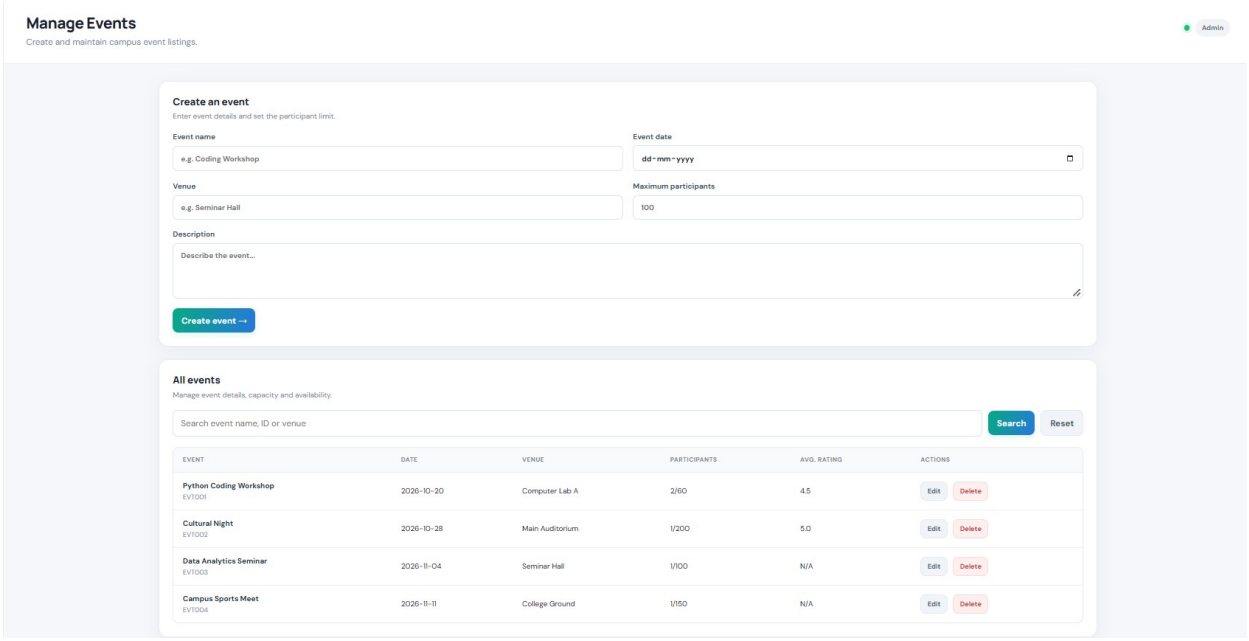
A.5 Administrator Dashboard

Event, registration and rating summary.



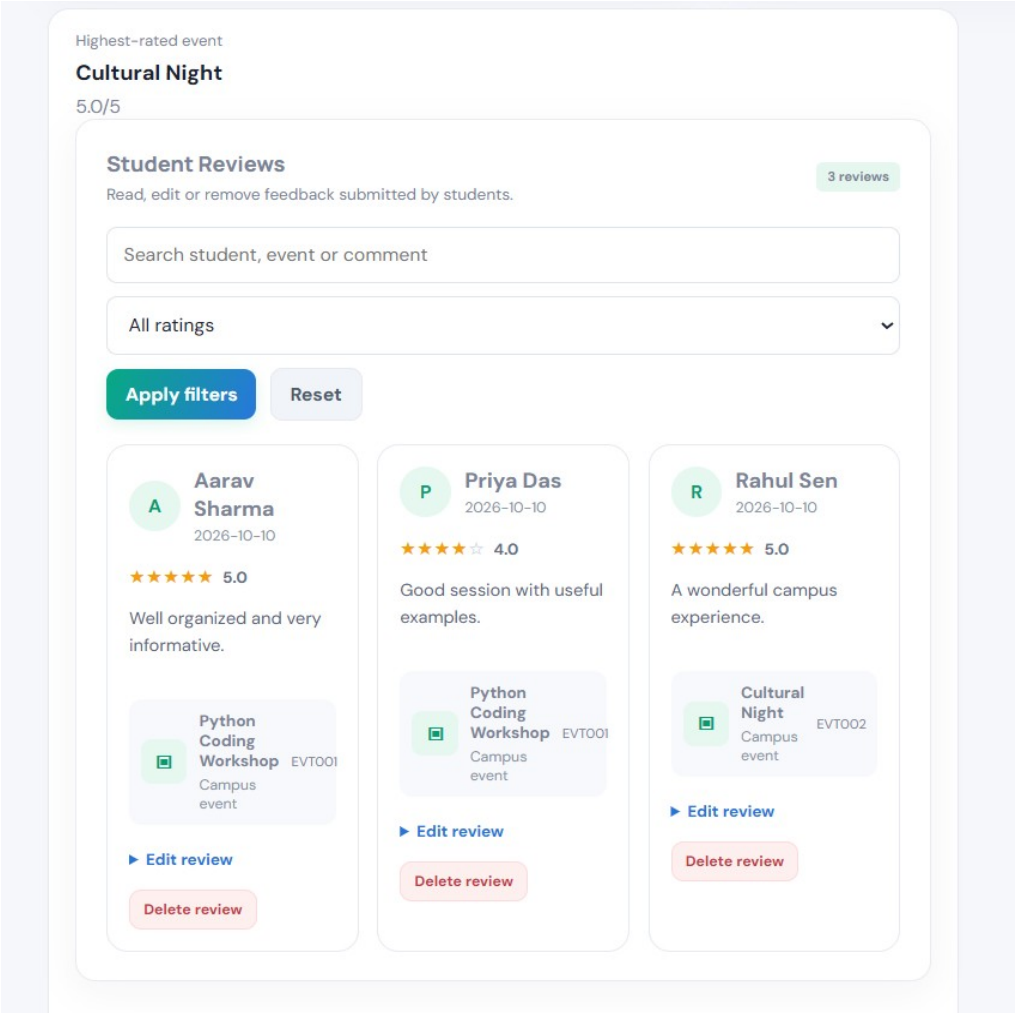
A.6 Manage Events and Students

Administrator event form, event list and student directory.



A.7 Feedback

Student review cards, edit/delete controls and analytics.



APPENDIX B: SAMPLE SOURCE CODE

B.1 Password Hashing

The authentication service creates a random salt and derives a SHA-256 password hash using PBKDF2.

```
salt = secrets.token_bytes(16)
digest = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt,
240000)
stored_hash = salt.hex() + "$" + digest.hex()
```

B.2 Password Validation

```
if len(password) < 8 or not re.search(r"[A-Z]", password) or not
re.search(r"[a-z]", password) or not re.search(r"\d", password):
    raise ValueError("Password must be at least 8 characters and include
uppercase, lowercase and a number.")
```

B.3 Registration Date and Capacity Validation

```
if event_date < date.today():
    raise ValueError("Registration is closed because this event date has
already passed.")

if any(r["student_id"] == student_id and r["event_id"] == event_id
       and r["status"] == "Registered" for r in rows):
    raise ValueError("You are already registered for this event.")

if capacity <= 0 or len(active_for_event(event_id)) >= capacity:
    raise ValueError("This event is full.")
```

B.4 Feedback Eligibility

```
if not any(r["student_id"] == student_id and r["event_id"] == event_id
       and r["status"] == "Registered" for r in regs):
    raise ValueError("You must have an active registration to submit
feedback.")

if any(r["student_id"] == student_id and r["event_id"] == event_id for r
in rows):
    raise ValueError("Feedback already submitted for this event.")
```

B.5 Start the Application

```
initialize_files()
refresh_charts()
server = ThreadingHTTPServer((HOST, PORT), Handler)
print(f"Open: http://{HOST}:{PORT}")
server.serve_forever()
```

APPENDIX C: CSV DATA STRUCTURE

File	Field	Description
students.csv	student_id	Unique student record identifier.
students.csv	name	Student's full name.
students.csv	email	Gmail address used for account login.
students.csv	student_code	Unique student code.
students.csv	password_hash	Salted PBKDF2-HMAC-SHA256 hash.
admins.csv	admin_id	Unique administrator record identifier.
admins.csv	name	Administrator name.
admins.csv	email	Administrator Gmail address.
admins.csv	admin_code	Administrator login code.
admins.csv	password_hash	Salted password hash.
events.csv	event_id	Unique event identifier.
events.csv	event_name	Name of the campus event.
events.csv	date	Event date in YYYY-MM-DD format.
events.csv	venue	Location of the event.
events.csv	max_participants	Maximum permitted active registrations.
events.csv	description	Description of the event.
registrations.csv	registration_id	Unique registration identifier.
registrations.csv	student_id / event_id	References the student and event.
registrations.csv	registration_date / status	Date of registration and Registered/Cancelled status.
feedback.csv	feedback_id	Unique feedback identifier.
feedback.csv	student_id / event_id	References reviewer and event.
feedback.csv	rating / comment	Integer rating from 1 to 5 and review text.
feedback.csv	feedback_date	Date the feedback was submitted.

APPENDIX D: TEST CASES

Test ID	Test condition	Expected / observed result	Status
TC-01	Valid student credentials	Student dashboard opens.	Implemented
TC-02	Incorrect password	Login is rejected with an error message.	Implemented
TC-03	Student opens administrator route	Administrator-only access is denied/redirected.	Implemented
TC-04	Event date is before today	Registration is rejected.	Implemented
TC-05	Event has reached maximum capacity	Registration is rejected as full.	Implemented
TC-06	Student registers twice for same event	Duplicate active registration is rejected.	Implemented
TC-07	Student cancels own registration	Registration status changes to Cancelled.	Implemented
TC-08	Student submits feedback without active registration	Feedback is rejected.	Implemented
TC-09	Student submits feedback twice	Second feedback submission is rejected.	Implemented
TC-10	Administrator edits feedback	Updated rating/comment is saved.	Implemented
TC-11	Administrator deletes feedback	Feedback record is removed.	Implemented
TC-12	Daily chart is requested by browser	Current chart route allowlist does not include feedback_daily.png.	Needs correction
TC-13	Run seed_demo.py with empty students.csv	create_student call lacks required password argument.	Needs correction

REFERENCES

- **Python Software Foundation. Python Documentation – http.server.** https://docs.python.org/3/library/http.server.html
- **Python Software Foundation. Python Documentation – csv.** https://docs.python.org/3/library/csv.html
- **Python Software Foundation. Python Documentation – hashlib.** https://docs.python.org/3/library/hashlib.html
- **Pandas Documentation:** https://pandas.pydata.org/docs/
- **NumPy Documentation:** https://numpy.org/doc/
- **Matplotlib Documentation**: https://matplotlib.org/stable/
- **MDN Web Docs**: HTML. https://developer.mozilla.org/en-US/docs/Web/HTML
- **MDN Web Docs:** CSS. https://developer.mozilla.org/en-US/docs/Web/CSS
- **Brainware University**: MCA project and course guidelines supplied for the academic project.
- **Campus Event Management project source package supplied for documentation review**: app.py, services/, models/, analytics/, templates/, static/, data/ and README.md.